

views::cache_last

Document #: P3138R0
Date: 2024-02-14
Project: Programming Language C++
Audience: SG9
SG1
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

§ 1 Abstract

This paper proposes the `views::cache_last` adaptor that caches the result of the last dereference of the underlying iterator. To support this adaptor, it also proposes a relaxation of 16.4.6.10 [\[res.on.data.races\]](#) for operations on input-only iterators.

§ 2 Motivation

The motivation for this view is given in [\[P2760R1\]](#) and quoted below for convenience:

One of the adaptors that we considered for C++23 but ended up not pursuing was what range-v3 calls `cache1` and what we'd instead like to call something like `cache_last`. This is an adaptor which, as the name suggests, caches the last element. The reason for this is efficiency - specifically avoiding extra work that has to be done by iterator dereferencing.

The canonical example of this is `transform(f) | filter(g)`, where if you then iterate over the subsequent range, `f` will be invoked twice for every element that satisfies `g`:

```
int main()
{
    std::vector<int> v = {1, 2, 3, 4, 5};

    auto even_squares = v
        | std::views::transform([](int i){
            std::print("transform: {}\n", i);
            return i * i;
        })
        | std::views::filter([](int i){
            std::print("filter: {}\n", i);
            return i % 2 == 0;
        });

    for (int i : even_squares) {
        std::print("Got: {}\n", i);
    }
}
```

prints the following (note that there are 7 invocations of transform):

```
transform: 1
filter: 1
transform: 2
filter: 4
transform: 2
Got: 4
transform: 3
filter: 9
transform: 4
filter: 16
transform: 4
Got: 16
transform: 5
filter: 25
```

The solution here is to add a layer of caching:

```
auto even_squares = v
| views::transform(square)
| views::cache_last
| views::filter(is_even);
```

Which will ensure that square will only be called once per element.

§ 3 Design

§ 3.1 Caching in `operator*`

This is the only reasonable place for this caching to happen. Caching in `operator++` would require unnecessary overhead on every traversal if the user does not need to dereference every iterator (e.g., a striding view).

§ 3.2 Relaxing `[res.on.data.races]`

Because `operator*` is required to be a `const` member function (the cost of relaxing that requirement would be prohibitive as it affects every iterator and range adaptor), yet we want to have it perform a potentially-modifying operation, our options are basically:

- Relax 16.4.6.10 [\[res.on.data.races\]](#) p3
- Require synchronization.

This paper proposes the former. Input-only iterators, in general, are poor candidates for sharing across threads because, even when copyable (and they are not required to be in C++20), they are subject to “spooky action at a distance” where incrementing one iterator invalidates all copies thereof. This is why parallel algorithms require at least forward iterators. Sharing references to the same input iterator across threads seems like a fairly contrived scenario.

Moreover, `std::istreambuf_iterator` already doesn't meet this requirement in [every major standard library implementation](#) yet it does not seem to have appeared on any standard library maintainer's radar in the decade since the publication of C++11. This further suggests that this guarantee is not relied upon in practice for input-only iterators.

It is true that we could require synchronization on `operator*() const`. This probably isn't terrible expensive in this context (we only need to protect against `operator*` calls racing with each other, not them racing with `operator++`), but adding synchronization to an adaptor whose primary purpose is to improve performance seems particularly dissatisfying when that synchronization will almost never be actually necessary.

§ 3.3 What's the reference type?

range-v3 uses `range_value_t<V>&&`, but this somewhat defeats the purpose of caching if you can so easily invalidate it. Moreover, there doesn't seem to be a reason to force an independent copy of the `value_type`. So long as the underlying iterator is not modified, the reference obtained from `operator*` should remain valid.

This paper therefore proposes `range_reference_t<V>&`. Note that even if the reference type is a real reference, it can be an *expensive-to-compute* reference, so caching could still make sense.

§ 3.4 Properties

`cache_last` is never borrowed, input-only, never common, and not const-iterable.

§ 3.5 `iter_move` and `iter_swap`

`indirectly_readable` and `indirectly_swappable` requires `iter_move` and `iter_swap` to respectively not modify `i` (in the 18.2 [\[concepts.equality\]](#) sense). Moreover, `indirectly_readable` requires `*i` to be equality-preserving. So the cache should not be invalidated by either operation. (The underlying element might be modified, but the reference itself, obtained from dereferencing the iterator, cannot.)

§ 4 Wording

This wording is relative to [\[N4971\]](#).

§ 4.1 Carve-out for input iterators from [\[res.on.data.races\]](#)

Edit 16.4.6.10 [\[res.on.data.races\]](#) p3 as indicated:

- ³ A C++ standard library function **other than a member or friend function of a non-forward input iterator** (25.3.4 [\[iterator.concepts\]](#), 25.3.5 [\[iterator.cpp17\]](#)) shall not directly or indirectly modify objects (6.9.2 [\[intro.multithread\]](#)) accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function's non-const arguments, including `this`.

§ 4.2 Addition to `<ranges>`

Add the following to 26.2 [\[ranges.syn\]](#), header `<ranges>` synopsis:

```
// [...]
namespace std::ranges {
    // [...]

    // [range.cache.last], to input view
    template<input_range V>
        requires view<V>
        class cache_last_view;

    namespace views {
        inline constexpr unspecified cache_last = unspecified;
    }
}
```

§ 4.3 `cache_last`

Add the following subclause to 26.7 [\[range.adaptors\]](#):

§ 26.7.? Cache last view [\[range.cache.last\]](#)

§ 26.7.?.1 Overview [\[range.cache.last.overview\]](#)

- 1 `cache_last_view` caches the last element of its underlying sequence so that the element does not have to be recomputed on repeated access. [*Note 1*: This is useful if computation of the element to produce is expensive. — *end note*]
- 2 The name `views::cache_last` denotes a range adaptor object (26.7.2 [\[range.adaptor.object\]](#)). Let `E` be an expression. The expression `views::cache_last(E)` is expression-equivalent to `cache_last_view(E)`. [*Drafting note*: Intentional CTAD to avoid double wrapping if `E` is already a `cache_last_view`.]

§ 26.7.?.2 Class template `cache_last_view` [\[range.cache.last.view\]](#)

```
template<input_range V>
    requires view<V>
class cache_last_view : public view_interface<cache_last_view<V>>{
    V base_ = V(); // exposition
    only
    using cache_t = conditional_t<is_reference_v<range_reference_t<V>>, // exposition
    only
                                add_pointer_t<range_reference_t<V>>,
                                range_reference_t<V>>;

    non-propagating-cache<cache_t> cache_; // exposition
    only

    class iterator; // exposition
    only
```

```

class sentinel; // exposition
    only

public:
    cache_last_view() requires default_initializable<V> = default;
    constexpr explicit cache_last_view(V base);

    constexpr V base() const & requires copy_constructible<V> { return base_; }
    constexpr V base() && { return std::move(base_); }

    constexpr auto begin();
    constexpr auto end();

    constexpr auto size() requires sized_range<V>;
    constexpr auto size() const requires sized_range<const V>;
};

template<class R>
    cache_last_view(R&&) -> cache_last_view<views::all_t<R>>;

```

```
constexpr explicit cache_last_view(V base);
```

1 *Effects:* Initializes *base_* with `std::move(base)`.

```
constexpr auto begin();
```

2 *Effects:* Equivalent to:

```
return iterator(*this);
```

```
constexpr auto end();
```

3 *Effects:* Equivalent to:

```
return sentinel(*this);
```

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

4 *Effects:* Equivalent to:

```
return ranges::size(base_);
```

§ 26.7.?.3 Class `cache_last_view::iterator` [range.cache.last.iterator]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
    class cache_last_view<V>::iterator {
        cache_last_view* parent_; // exposition only
        iterator_t<V> current_; // exposition only

        constexpr explicit iterator(cache_last_view& parent); // exposition only

    public:

```

```

using difference_type = range_difference_t<V>;
using value_type = range_value_t<V>;
using iterator_concept = input_iterator_tag;

iterator(iterator&&) = default;
iterator& operator=(iterator&&) = default;

constexpr iterator_t<V> base() &&;
constexpr const iterator_t<V>& base() const & noexcept;

constexpr range_reference_t<V>& operator*() const;

constexpr iterator& operator++();
constexpr void operator++(int);

friend constexpr range_rvalue_reference_t<V> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<V>>;
};
}

```

```
constexpr explicit iterator(cache_last_view& parent);
```

- 1 *Effects:* Initializes `current_` with `ranges::begin(parent.base_)` and `parent_` with `addressof(parent)`.

```
constexpr iterator_t<V> base() &&;
```

- 2 *Returns:* `std::move(current_)`.

```
constexpr const iterator_t<V>& base() const & noexcept;
```

- 3 *Returns:* `current_`.

```
constexpr iterator& operator++();
```

- 4 *Effects:* Equivalent to:

```

++current_;
parent_ -> cache_.reset();
return *this;

```

```
constexpr void operator++(int);
```

- 5 *Effects:* Equivalent to: `++*this;`

```
constexpr range_reference_t<V>& operator*() const;
```

- 6 *Effects:* Equivalent to:

```

if constexpr (is_reference_v<range_reference_t<V>>) {
    if (!parent_ -> cache_) {

```

```

        parent_ -> cache_ = addressof(as-lvalue(*current_));
    }
    return **parent_ -> cache_;
}
else {
    if (!parent_ -> cache_) {
        parent_ -> cache_.emplace-deref(current_);
    }
    return *parent_ -> cache_;
}

```

```

friend constexpr range_rvalue_reference_t<V> iter_move(const iterator& i)
    noexcept(noexcept(ranges::iter_move(i.current_)));

```

7 *Effects:* Equivalent to: `return ranges::iter_move(i.current_);`

```

friend constexpr void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<V>>;

```

8 *Effects:* Equivalent to: `ranges::iter_swap(x.current_, y.current_);`

§ 26.7.3.3 Class `cache_last_view::sentinel` [`range.cache.last.sentinel`]

```

namespace std::ranges {
    template<input_range V>
        requires view<V>
    class cache_last_view<V>::sentinel {
        sentinel_t<V> end_ = sentinel_t<V>(); // exposition only

        constexpr explicit sentinel(cache_last_view& parent); // exposition only

    public:
        sentinel() = default;

        constexpr sentinel_t<V> base() const;

        friend constexpr bool operator==(const iterator& x, const sentinel& y);
    };
}

```

```
constexpr explicit sentinel(cache_last_view& parent);
```

1 *Effects:* Initializes `end_` with `ranges::end(parent.base_)`.

```
constexpr sentinel_t<V> base() const;
```

2 *Returns:* `end_`.

```
friend constexpr bool operator==(const iterator& x, const sentinel& y);
```

3 *Returns:* `x.current_ == y.end_;`

§ 5 References

[N4971] Thomas Köppe. 2023-12-18. Working Draft, Programming Languages — C++. <https://wg21.link/n4971>

[P2760R1] Barry Revzin. 2023-12-15. A Plan for C++26 Ranges. <https://wg21.link/p2760r1>